GRAPHIC ERA

PROJECT-BASED LEARNING

PHASE-I EVALUATION

MiniVCS: A Mini Version Control and Collaborative Repository Management System

Team ID: DSCPP-III-2026-T015

Department of Computer Science & Engineering — Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T015
Semester: 3rd
Sections: ML4, ML2, ML1, DS1
Course: Data Structure and Programming in C++

Mentor

Ms. Meenakshi Maindola
Email: meenakshi.maindola@geu.ac.in
Interaction completed: 2

Team Members

1. Rashi Sharma — Leader
Roll No: 2028471
Section: ML4
Student ID: 2510382287

2. Dhanvi Sarpal
Roll No: 2028708
Section: ML2
Student ID: 2510370068

3. Avni Srivastava
Roll No: 2028683
Section: ML1
Student ID: 2510370007

4. Aditi Sharma
Roll No: 2028618
Section: DS1
Student ID: 2510370447

MiniVCS: A Mini Version Control and Collaborative Repository Management System

Problem Statement and Motivation

Problem Statement

Managing multiple versions of files manually becomes difficult when changes are frequent and several users work on the same project. There is a need for a simple system that can store, track and manage different versions of files efficiently.

Motivation

- Maintain different versions of files.
- Track changes made by users.
- Organize repository information.
- Retrieve previous versions easily.
- Demonstrate practical DSA applications.

Target Users

- Students
- Developers
- Small project teams
- Software learners
- Academic evaluators

Objectives

Primary Objectives

1. Develop a mini version control system.
2. Create and manage repositories.
3. Track files and their changes.
4. Store multiple versions efficiently.
5. Maintain commit history.

System Objectives

6. Provide version retrieval.
7. Support rollback/restoration.
8. Apply Data Structures in C++.
9. Apply OOP concepts.
10. Provide efficient repository management.

Key Objective

Efficiently manage, track and retrieve different versions of project files using Data Structures and Object-Oriented Programming.

Proposed Solution

Working

1. User creates a repository.
2. Files are added to the repository.
3. Changes in files are tracked.
4. User creates a commit.
5. Commit information is stored.
6. A new version is created.
7. User can view version history.
8. Previous versions can be restored.

Main Features

- **Repository Initialization**
- **File Tracking**
- **File Addition**
- **Commit Management**
- **Version History**
- **Status Checking**
- **Version Retrieval**
- **Rollback / Restoration**
- **Repository Management**

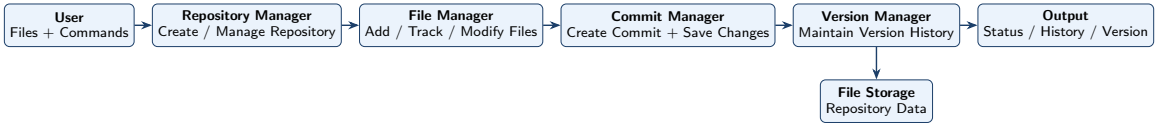
Core Idea

Store repository information and file versions in an organized structure so that changes can be tracked and previous versions can be retrieved whenever required.

Integration of PBL Course Concepts

Data Structure / Concept	Application in MiniVCS
Arrays	Store collections of files, commits and repository information.
Linked Lists	Maintain sequential commit and version history.
Stacks	Support undo, rollback and restoration of previous versions.
Queues	Manage pending operations and sequential tasks.
Trees	Represent repository and directory structures.
Hashing	Provide fast file identification and lookup.
Searching	Search files, commits and repository information.
Sorting	Arrange commits, files and version information.
OOP in C++	Classes, objects, encapsulation, inheritance and modular design.
File Handling	Store repository data and file versions persistently.

System Architecture / Workflow



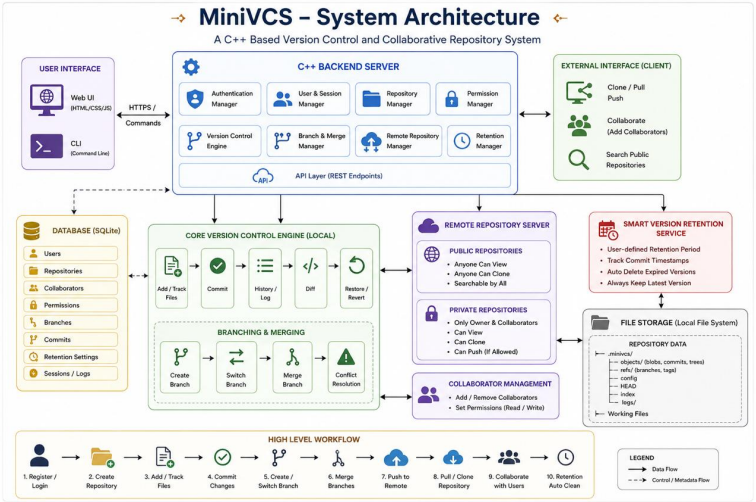
Complete Workflow

Create Repository → Add Files → Track Changes → Create Commit → Store Version → View History → Restore Previous Version

Data Flow

User Input → Repository Management → File Tracking → Commit Management → Version History → File Storage → Output

Detailed System Architecture Diagram



Technology Stack and Methodology

Technology Stack

- **Programming Language:** C++
- **IDE:** VS Code / Visual Studio
- **Storage:** File System
- **Version Control:** Git / GitHub
- **Design:** Object-Oriented Programming
- **Core:** Data Structures

Methodology

1. Requirement Analysis
2. System Design
3. Data Structure Selection
4. Module Development
5. Repository Implementation
6. Testing
7. Integration
8. Final Evaluation

Initial Progress and Team Contribution

Initial Progress

- Project problem and scope identified.
- Basic system workflow designed.
- Required Data Structures identified.
- Technology stack selected.
- Core modules planned.
- Repository and version management concepts studied.

Member	Role	Contribution
Rashi Sharma	Team Lead	Planning and Coordination
Dhanvi Sarpal	Developer	Core Implementation
Avni Srivastava	DSA / Testing	Data Structures and Testing
Aditi Sharma	UI / Documentation	Documentation and Interface

Roadmap and Expected Outcomes

Roadmap

Phase I	Phase II	Phase III
Problem Identification	Core Development	Integration
System Design	Repository Management	Testing
Data Structure Selection	Version Tracking	Final Evaluation

Expected Outcomes

- Working MiniVCS system.
- Efficient file and repository management.
- Commit and version history.
- Version retrieval and restoration.
- Practical implementation of DSA and OOP.
- Foundation for future collaborative features.

References

- Thomas H. Cormen et al., *Introduction to Algorithms*.
- Bjarne Stroustrup, *The C++ Programming Language*.
- Git Documentation and Version Control Concepts.
- Data Structures and Algorithms course material.
- Object-Oriented Programming concepts in C++.

Thank You

Questions & Suggestions?

MiniVCS

A Mini Version Control and Collaborative Repository Management System